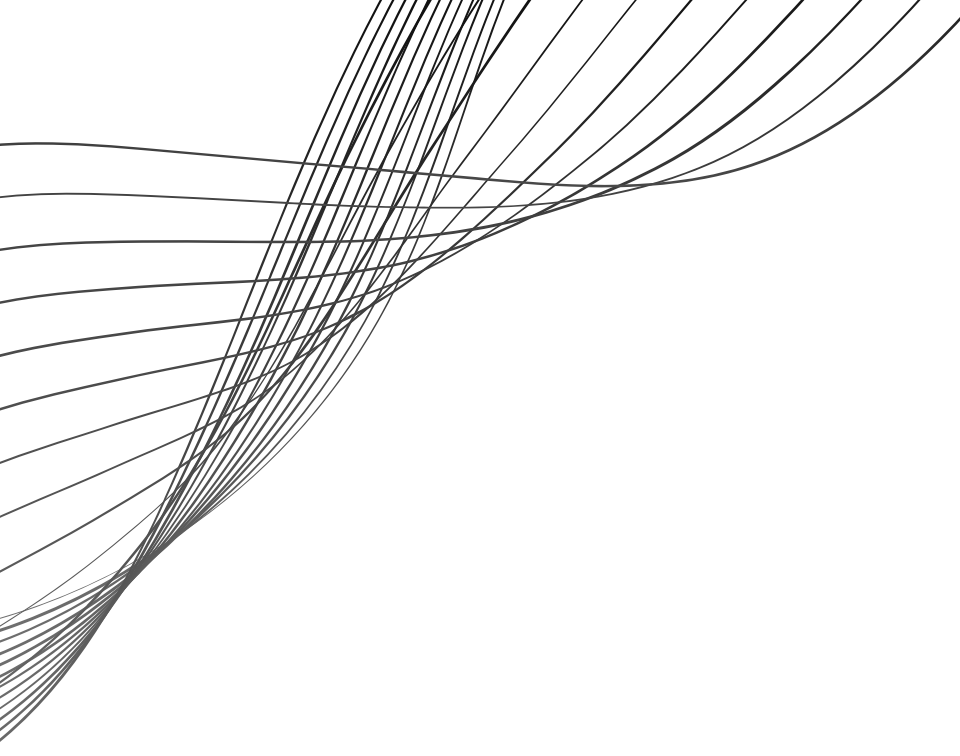


POO COM C#

Pedro Henrique Ribeiro Dias

Inatel

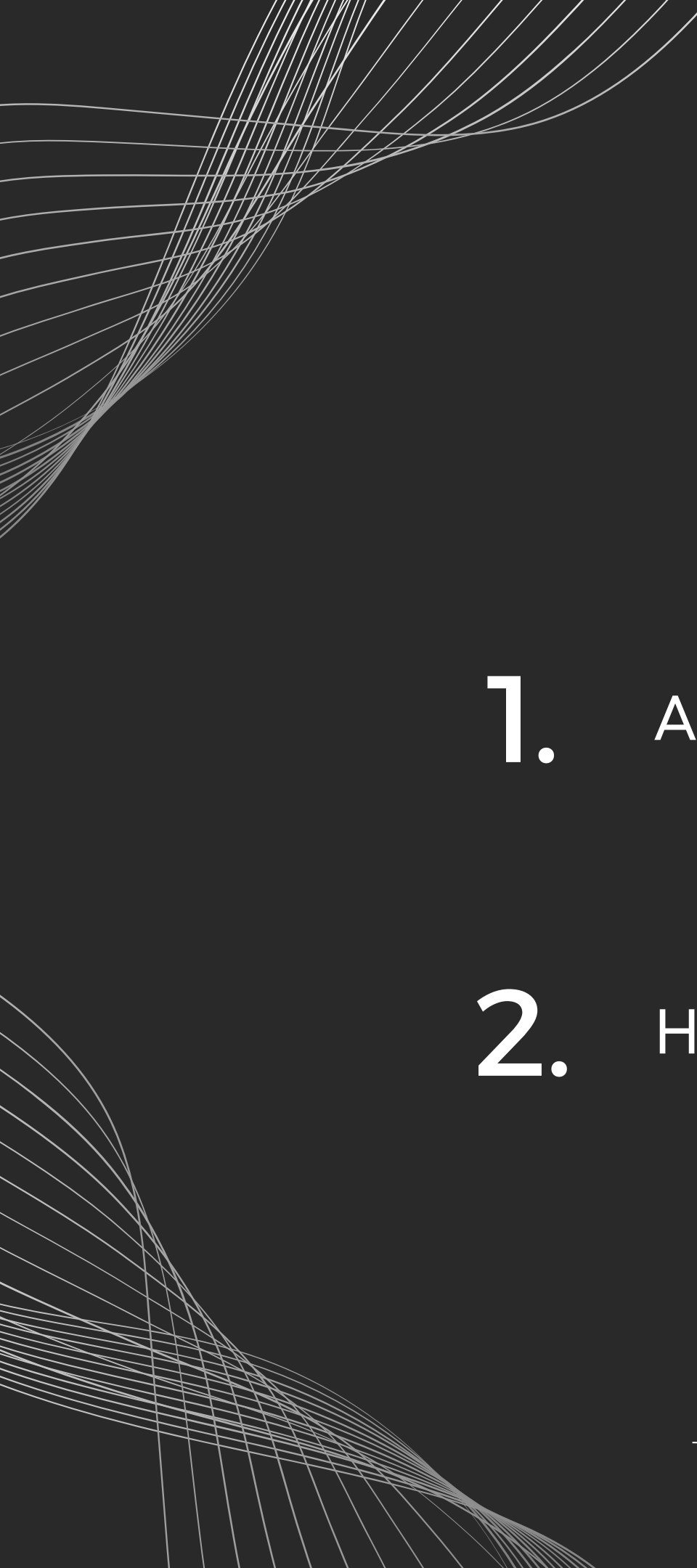


INTRODUÇÃO

Enquanto o C++ exige controle manual de memória, o C# automatiza essa tarefa com o Garbage Collector, além de apresentar uma sintaxe mais limpa e simplificada, especialmente no uso de propriedades para encapsulamento.

POO COM C#

1. Permite melhor organização, reutilização de código e manutenção facilitada, promovendo sistemas altamente escaláveis e flexíveis.
2. Criamos classes como moldes para objetos que possuem atributos e métodos, ou seja, características e comportamentos



PILARES DA POO

1. Abstração

2. Herança

3. Encapsulamento

4. Polimorfismo



ASSOCIAÇÕES

RELACIONAMENTOS

1. Agregação

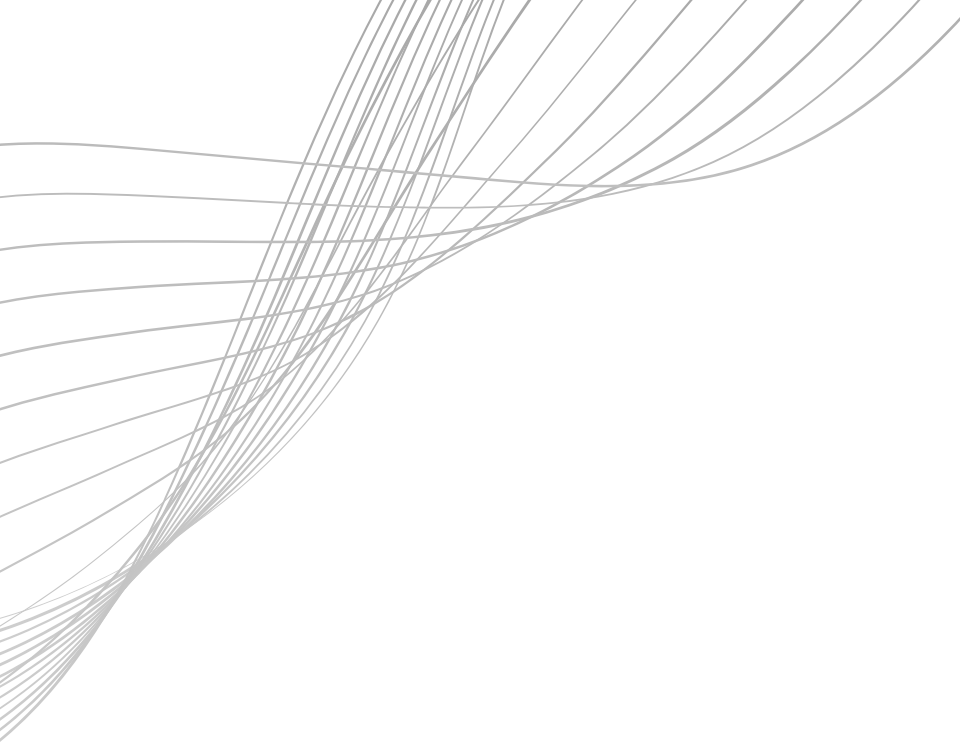
2. Composição

Relacionamentos entre Classes

AGREGAÇÃO

Agregação (Relacionamento "Tem"):

- É um tipo de Composição mais fraco.
- Representa uma relação "todo-parte" onde as partes podem existir independentemente do todo.
- Exemplo: Um Carro agrega um Motor. O motor pode ser removido e usado em outro carro.



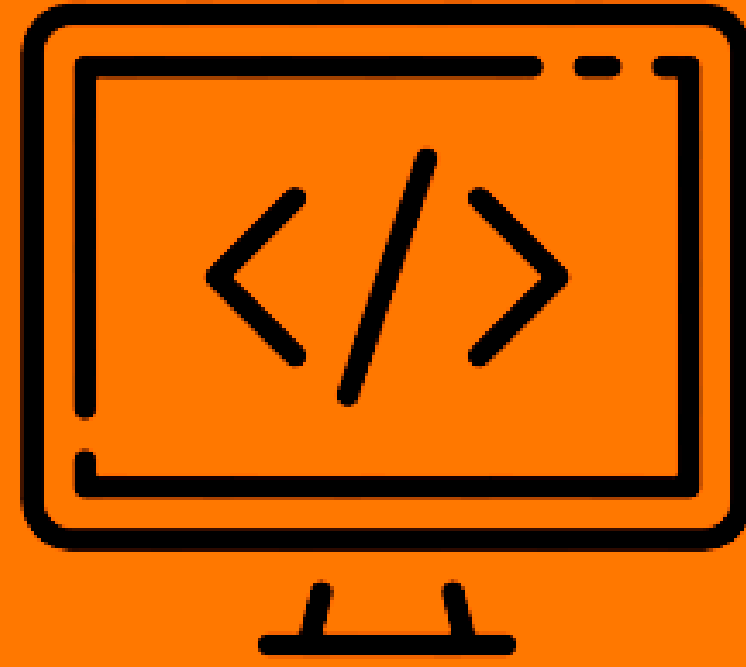
COMPOSIÇÃO

Composição (Forte): A parte não pode existir sem o todo.

- Exemplo: Um Carro compõe um Chassi. Se o carro for destruído, o Chassi não sobrevive sozinho.

CONSTRUTORES E COLECTIONS

1. Construtores:
 - Método automático que inicializa o objeto.
 - Garante que o objeto nasça em um estado válido.
 - Chamado usando new.
2. Coleções (Listas e Vetores):
 - Usadas para armazenar vários objetos.
 - List<T>: Coleção dinâmica (tamanho variável).
 - Polimorfismo: Permite armazenar objetos de diferentes classes filhas em uma lista do tipo da classe mãe (List<Pai>).



**KEEP
CALM**

AND

**PROGRAMME
ORIENTADO A OBJETOS**

EXERCÍCIO 1

Cenário: Em O Senhor dos Anéis, durante o cerco a Minas Tirith, cada combatente convocado é registrado com nome, povo e posto. Alguns chegam armados, outros não.

Tarefa:

1. Crie a classe `CombatenteDeGondor` com as propriedades `Nome`, `Povo` e `Posto`, todas com `private set` e preenchidas pelo construtor.
2. Crie a propriedade `Armamento`, com `private set` e valor inicial `"Desarmado"`, e o método `Equipar(string arma)` para alterá-la.
3. Crie o método `ApresentarUnidade()`, que exibe os dados do combatente. O armamento só deve aparecer se for diferente de `"Desarmado"`.
4. Na `Main`, crie três combatentes à sua escolha, equipe pelo menos um e chame `ApresentarUnidade()` em todos.
5. Tente alterar o `Posto` de um combatente na `Main`, observe o erro e comente a linha.

EXERCÍCIO 2

Cenário: No mundo Pokémon, numa batalha de exibição, cada Pokémon entra em campo e ataca do seu jeito: os do Tipo Planta usam um golpe próprio, os do Tipo Elétrico atacam normalmente e depois soltam uma descarga, e os sem tipo definido usam um ataque comum.

Tarefa:

1. Crie a classe base Pokemon com Especie e Nivel (recebidos no construtor) e o método virtual Atacar(), com um ataque genérico.
2. Crie as classes TipoPlanta e TipoEletrico, que herdaram de Pokemon.
3. Em TipoPlanta, sobrescreva Atacar() sem chamar o pai. Em TipoEletrico, sobrescreva Atacar() chamando primeiro base.Atacar().
4. Na Main, crie uma List<Pokemon> com um Pokémon de cada classe (você escolhe quais) e percorra-a com foreach chamando Atacar().

EXERCÍCIO 3

Cenário: A elfa Frieren carrega um grimório especial criado junto com ela (composição) e viaja com companheiros que encontrou ao longo da jornada, existindo antes de conhecê-la (agregação).

Tarefa:

1. Crie a classe Grimorio com a propriedade FeiticoFavorito (valor inicial "Nenhum") e o método Abrir(), que a exibe.
2. Crie a classe Companheiro com Nome e Funcao e o método Apresentar().
3. Crie a classe Maga com:
 - Nome e um Grimorio criado dentro do construtor (Composição);
 - uma lista privada de Companheiro, o método Recrutar(Companheiro c) (Agregação) e o método MostrarGrupo().
4. Na Main, crie dois companheiros antes da Maga, crie a Maga, recrute-os, defina o feitiço favorito do grimório e chame MostrarGrupo() e Abrir().
5. Explique, em um comentário no código, o que é composição e o que é agregação no seu programa.

EXERCÍCIO 4

Cenário: No universo de Lovecraft, na Biblioteca da Universidade Miskatonic, um pesquisador cataloga relatos de entidades cósmicas. Algumas têm origem conhecida; outras, não.

Tarefa:

1. Crie a classe base `EntidadeCosmica` com `Nome` e `Origem` (valor inicial "Desconhecida") e o método virtual `Manifestar()`, que só exibe a origem se ela for conhecida.
2. Crie as classes `Profundo` e `MiGo`, que herdam de `EntidadeCosmica`. Uma sobrescreve `Manifestar()` sem chamar o pai; a outra chama `base.Manifestar()` antes.
3. Crie a classe `Pesquisador` com `Nome`, uma lista privada de `EntidadeCosmica`, o método `Catalogar(EntidadeCosmica e)` e o método `LerCatalogo()`.
4. Na `Main`, crie uma entidade de cada classe, defina a origem de pelo menos uma, catalogue todas em um pesquisador e chame `LerCatalogo()`.

FIM DA AULA!

zSh3rl0cK/**S01-
monitoria**



 1
Contributor

 0
Issues

 0
Stars

 0
Forks



zSh3rl0cK/S01-monitoria

Contribute to zSh3rl0cK/S01-monitoria development by creating an account on GitHub.

 GitHub